2024 33rd International Conference on Computer Communications and Networks (ICCCN) | 979-8-3503-8461-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICCCN61486.2024.10637611

Edge Computing Network Intrusion Detection System in IoT Using Deep Learning

Andres Hinojosa, Nahid Ebrahimi Majd
Department of Computer Science and Information Systems
California State University San Marcos, United States
hinoj030@csusm.edu, nmajd@csusm.edu

***Abstract*— A Network Intrusion Detection System (NIDS) is a technology to monitor the network traffic and detect and classify the possible cyber threats towards a computer network. In case it detects a potential cyberattack, it will alert the network prevent and defeat security system. Identifying the type of attack is invaluable for such system to block the malicious traffic or mitigate its impacts on the network. The Internet of Things (IoT) is a system of devices connected to the Internet, mainly for the purpose of acquiring data and transmitting that to the cloud, where the data will be processed, analyzed, and stored on the cloud and used by the user. In many cases, a group of IoT devices located at the same place form a network and send their data to the cloud through the network's gateway. As IoT technology is rapidly growing, it is crucial to develop more accurate and efficient intrusion detection and classification systems. In this paper, we propose efficient deep learning models that classify the network traffic to benign vs seven main types of IoT attacks with a focus on CICIoT2023 dataset at the edge of IoT network by leveraging edge computing. This research addresses the challenges of applying deep learning on intrusion detection datasets, including data imbalance and data distribution discrepancy using data preprocessing techniques, such as random under sampling, class weighting, and feature scaling. We also study the impact of different feature selection techniques on the efficiencies of our models. Our experimental results showed that our 1D-CNN model outperforms the state-of-the-art with 93.8% F1-score.**

Keywords*— *Network Intrusion Detection System; Edge Computing; Network security; Network Attack Classification; Deep Learning;

I. Introduction

In the modern life, the number of Internet of Things (IoT) devices is rapidly growing. A forecast by CISCO research team estimates that there will be 75.3 billion active IoT devices by 2025. The International Data Corporation (IDC) estimates that in 2025 IoT devices will generate almost 79.4 zettabytes (ZB) of data, which will be mainly sent to the cloud for further processing, analysis, and storage. IoT devices are often characterized as low-cost products with constrained computational power, memory, and software, which impose limited security solutions and updating mechanisms. This results in significant security concerns and vulnerabilities in IoT devices, making them one of the main targets of malwares that automatically infect them for the purpose of conducting network attacks, especially Botnet and DDoS attacks. Such attacks disrupt businesses and individuals' work and impose significant financial burden to the society. These network attacks require massive number of always-on machines taken over by an attacker, and IoT devices with all their security vulnerabilities are great choices to be exploited in such attacks. Besides that, due to their weak security solutions, IoT devices could be easily infected by any type of malware, e.g., Ransomware and Spyware, and automatically spread that to the network and infect other devices as well. Considering that most IoT devices are autonomous machines with the capacity to fulfill their tasks without any need to human interactions, even the IoT device owner may not know that the device is compromised and being exploited to spread a malware or conduct an attack because that does not impact the device's expected functionality. A compromised IoT device may keep acquiring the requested data and sending that to the cloud without recognizing that it is also exchanging malicious traffic with the network.

A Network Intrusion Detection System (NIDS) inspects the network traffic to detect and classify the malicious traffic exchanged with the network machines, whether they are the target or the source of the attack. Recently, machine learning and deep learning have been widely used by the networking research community to develop a new generation of NIDS. Due to unique features of IoT, e.g., the massive number and variety of IoT devices, their limited computation, capacity and poor security solutions, it is essential to design and develop an efficient NIDS tailored for IoT. There are two main approaches to develop a NIDS: (1) anomaly detection, (2) attack classification.

An anomaly detection NIDS is a system that detects the malicious traffic exchanged between the network and the cloud. On the other hand, an attack classification NIDS not only detects the malicious traffic, but also identifies the attack type. To efficiently protect the network against the cyberattacks, it is very useful to know the type of the conducted attack. That helps the network prevent and defeat system intercept the malicious traffic and mitigate the attack impacts.

A machine learning anomaly detection NIDS is a binary classifier, which detects the benign vs. malicious traffic. There are two approaches to implement such machine learning model: (1) train the model with labeled instances of both benign and malicious traffic. The model will classify a new instance of traffic to one of these two classes; (2) train the model with only benign instances. The model will detect the benign instances, and any instance that the model does not detect as benign will be identified as malicious traffic.

A machine learning attack classification NIDS is a multi-class classifier, which identifies the type of traffic, whether it is benign or a specific attack. There are two approaches to implement such machine learning model: (1) design a 2 step model. At step 1, a binary classifier detects benign vs. malicious traffic. If the traffic is identified as malicious, it will be passed to step 2, which is a multi-class classifier that identifies the attack type; (2) design an integrated model that classifies benign vs. the attack type. The second approach involves less

processing and delay, which is more efficient for an IoT NIDS. We will use the second approach to design our model.

An IoT device typically sends its acquired data to the cloud to be processed and stored. However, that transmitted data is usually not the malicious traffic that an IoT device may communicate in an attack scenario. Thus, although the cloud has the computing power to run a strong security solution, it is not the correct place to deploy an IoT NIDS. It is hard to capture the malicious traffic that is exchanged with a particular device if NIDS is implemented on the cloud. The correct place to capture and inspect such traffic is at the network edge. Edge computing is a relatively new paradigm. The idea of this model is to affload the data processing from the cloud to the network edge near the IoT end systems. The NIDS is attached to the network gateway and will receive all traffic exchanged between the network IoT devices and the cloud. It will execute the required pre-processing on the IoT device's network traffic and send the processed data to the NIDS for detection and classification. Then, it will get the classification result and send it to network prevent and defeat system for the next steps, including intercepting the malicious traffic and mitigating its negative impacts.

In an edge computing model, the NIDS is deployed at the network edge and will directly receive the IoT device's traffic from the gateway, run the required pre-processing on the data stream, feed the machine learning model with the processed data, gets the classification results from the model, and if the traffic is classified as a type of attack, it will directly send it to the network prevent and defeat system to take appropriate action. This model effectively offloads the security solution from the cloud to the edge, which results in significantly less processing load on the IoT device and the cloud and less latency that would be involved if the network needed to transmit the security inspection data to the cloud and wait for the cloud's response. In this model, the IoT device and the cloud perform their regular tasks and the edge will take care of all tasks related to attack detection and defense. The NIDS at the edge will track the network's IoT device's traffic, process the data streams, classify them, and take actions to defeat the potential attacks.

Another main advantage of an edge computing NIDS is data privacy. Any information about malicious traffic exchanged with the network remains in the network and potentially can be resolved in the network by the network's prevent and defeat system. The structure of this model contains 3 layers.

1. **The cloud:** This is the top layer, which receives the IoT device's acquired data and processes and stores them.
2. **The edge:** This is the medium layer where the NIDS is placed. The NIDS keeps track of any traffic exchanged between the IoT devices and the cloud or other machines. It processes the tracked data streams and classifies them. If it detects a network attack, it will report that to the network prevent and defeat system at the edge to take action and protect the network's IoT devices and the cloud.
3. **The IoT device:** This is the bottom layer. The IoT devices acquire data and send it to the cloud. The IoT devices are grouped in an IoT network based on their location, ownership, use cases, etc.

The main contributions of our research are as follows.

1. We proposed **CNN-based deep learning** models for an edge computing NIDS solution to detect and classify network attacks. Our models classify benign vs. 7 main types of network attacks.
2. We used the CICIoT2023 dataset to develop our models. This dataset includes a variety of attacks and has been created by conducting these attacks on a large scale IoT network. This dataset is imbalanced. **To combat the data imbalance issue**, we applied different pre-processing techniques, including random under sampling to downsize the majority classes and class weighting to balance the classes' weights according to their populations.
3. We studied the impact of different **feature selection techniques** on classifying network traffic in our models. We analyzed the efficiencies of our models. Our experimental results indicated that our models outperform the state-of-the-art.

The rest of this paper is organized as the following. Section 2 describes the related work. Section 3 explains the methodology, including the dataset and preprocessing. Section 4 describes the proposed deep learning models and their hyperparameters. Section 5 presents the results and discussion. Section 6 draws the conclusion.

II. Related Work

In our study, we focus on evaluating the performance of an intrusion detection system using the CICIoT2023 dataset.

The CICIoT2023 [1] is a publicly available dataset created in 2023 by Canadian Institute for Cybersecurity. It contains a variety of network attacks conducted on a large scale IoT network. It consists of data on benign and 7 categories, including 33 subcategories of network attacks. The 7 categories of attacks are DDoS, DoS, Recon, Web-Based, Brute Force, Spoofing, and Mirai. Since it includes a variety of different attacks, it has gained attention in the research community and has been used in several research studies. We review the most related ones here.

Neto, et al. [2] created this dataset and proposed Machine Learning (ML) and Deep Learning (DL) classifiers trained by this dataset. They created a large-scale network composed of 105 IoT devices where 7 devices were in charge of conducting attacks and malicious activities in the experiments. They used a variety of tools to conduct different types of attacks. They captured the network traffic using Wireshark and applied feature engineering to aggregate the data belong to each data stream. The dataset contains 46 aggregated features and a timestamp for each instance. The dataset is imbalanced. Most of the instances belong to DDoS, DoS, and Mirai categories while relatively few instances belong to other categories including benign traffic. Since the dataset is imbalanced, F1-score is a more precise metric to measure the model's performance than accuracy. They proposed ML and DL models to classify the attacks. Their Random Forest (RF) and Deep Neural Network (DNN) models provided the best performances. They designed classifiers for 3 scenarios as follows.

1. 2-classes: benign vs. attack
2. 8-classes: benign vs. 7 categories of attacks
3. 34-classes: benign vs. 33 subcategories of attacks

We first present the ML learning models proposed for this dataset [2], [3], [4], and then the DL models [2], [6], [7], [8].

The RF models proposed by [2] presented F1-scores of 96.53% for binary, 71.92% for 8-classes, and 71.4% for 34-classes classifications. To increase the performance, Narayan et al. [3] explored a variety of feature selection and oversampling methods. Feature selection is a technique to select the features that have the strongest correlation with the class. Removing the features that have weak correlation with the class reduces the noise and results in a more accurate model. To combat the data imbalance challenge, they applied oversampling techniques to the dataset. Oversampling is a technique that synthetically generates new samples for a minority class. This technique augments the dataset's minority classes, which results in more accurate classifications in such classes. In their best performing models, they used CfsSubsetEval (CFS) feature selection method, selecting 6 out of 46 features using Weka tool, and Balanced Random Forest Classifier sampling (BRFC) oversampling technique. With these techniques, they achieved F1-scores of 95.92 for binary,81.86% for 8-class, and 76.03% for 34-class RF classifiers. Wardhani et al. [4] used a different feature selection technique named MDI [5] and selected 6 most significant features consisting of 'IAT', 'Magnitude', 'Total Size', 'Min', 'Flow Duration', and 'Total Sum'. They proposed a blended ensemble learning model of Extra Tree, Random Forest, and Gradient Boosting algorithms on this dataset. This model uses soft voting where each of the three base models compute the predicted probabilities of all classes, and the output is the class with the greatest average probability. They proposed an 8-class ML classifier with F1-score of 99.07%.

Next, we discuss the DL models proposed for this dataset. Table 1 summarizes these models and their performances. [2] proposed DNN models with F1-scores of 94.03% for binary, 69.73% for 8-class, and 67.23% for 34-class classifiers. Abbas et al. [6] proposed a federated learning binary classifier. They defined 2 clients where each has a portion of the data and sends it to the server to train the model. The server uses this data to create a DNN model. They used SMOTE oversampling technique to overcome the data imbalance challenge. Their binary classifier achieved 99% F1-score. Abbas et al. [7] proposed a variety of DL models to classify 34 classes, benign vs. 33 subcategories of attacks. Their Recurrent Neural Network (RNN) model presented the best performance with 95.73% F1-score. Jony et al. [8] proposed a Long short-term memory (LSTM) model to classify 34 classes, benign vs. 33 subcategories of attacks. They achieved a remarkably high F1-score of 99%.

Table 1: Deep Learning classifiers trained by CICIoT2023

	model	2-classes F1-score	8-classes F1-score	34-classes F1-score
[2]	DNN	94.03	69.73	67.23
[6]	FL-DNN	99.00	-	-
[7]	RNN	-	-	95.73
[8]	LSTM	-	-	98.59

A few other research studies used different approaches to combat the data imbalance. For example, Wnag et al. [9] selected a few subcategories of attacks with more balanced distribution of instances composing of benign, 4 subcategories of Recon, and 3 subcategories of Mirai. They proposed a bidirectional LSTM (BiLSTM) model on their extracted dataset. Their model achieved 92% F1-score. Nkoro et al. [10] combined the related minority classes together to create a class with higher number of instances and make a relatively more balanced dataset. They proposed DL models to classify 5 classes, benign vs. DDoS, DoS, Enumeration, and Malware. They grouped attacks such as Backdoor, password, MITM, ransomware, Uploading, XSS (cross-site scripting), and SQL injection in one class and named it malware. They grouped the remaining attacks including fingerprinting, enumeration, port scanning, and vulnerability scanner in another class, and named it enumeration. Their best performing model is a CNN-BiLSTM with 99% F1-score. We did not include the last two models in Table 1 because their sets of classes are different from the original dataset.

The related research has studied a variety of techniques to reduce the negative impacts of data imbalance on DL models trained by this dataset. They proposed DL binary classifiers, 34-class classifiers, and other classifiers. However, no effective 8-class DL classifier has been proposed for this dataset yet. One main reason is the large gaps between the sizes of these 8 classes in the dataset, which makes the dataset highly imbalanced. In our research, we fill this research gap by applying techniques to tackle data imbalance and proposing an efficient DL model that classifies 8-classes: benign vs. 7 categories of attacks.

III. Methodology

A. Dataset

We used CICIoT2023 [1] to train our models. This dataset contains instances on benign and 7 main categories of IoT attacks, containing DDoS, DoS, Recon, Web-Based, Brute Force, Spoofing, and Mirai. Each instance is labeled with its category type. Each instance of the dataset is aggregated data extracted from a data stream between a source and a destination. Each instance has a timestamp and 46 aggregated features. We removed the timestamp as it is irrelevant to the traffic type. The other 46 features are numeric. Some of them are a value, e.g., the flow's transmission rate, the number of packets in the flow, and the minimum packet length in the flow; some are binary, e.g., whether the SYN flag is set; and the others are an integer that identifies a category, e.g., a pre-defined integer that identifies the protocol type.

Table 2: Number of instances in each category

	Category	Instances	Percentage
1	DDoS	33,984,560	72.79%
2	DoS	8,090,738	17.33%
3	Mirai	2,634,124	5.64%
4	Benign	1,098,195	2.35%
5	Spoofing	486,504	1.04%
6	Recon	354,565	0.76%
7	Web	23,157	0.05%
8	BruteForce	12,191	0.03%

B. Data imbalance

Table 2 demonstrates the number of instances in each category. This dataset is highly imbalanced and there are large gaps between the number of instances in different categories. While more than 90% of the data are in DDoS and DoS categories, only less than 1% of the data is in Recon, Web, and BruteForce categories. Data imbalance can negatively impact the model's performance. Besides that, when there is a large gap between the sizes of two classes in the dataset, the accuracy of the trained model could be misleading. The model may show high accuracy, but it could be very accurate in classifying the dominant classes while very inaccurate in classifying the minority classes. The reason is in such a case, the model is well trained to classify the dominant instances, however, it has had very few instances to learn the patterns of minority classes. Thus, it has very low performance in classifying the minority classes. To mitigate this negative impact, F1-score is a better metric to reflect the model's performance. However, still the main problem is the large gaps between the sizes of the classes.

Table 3: The dataset after random under sampling

Category	Sub-category	Rows	Total	%
DDoS	PSHACK_Flood	10,000	120,000	38.41%
	ICMP_Flood	10,000		
	UDP_Flood	10,000		
	ICMP_Fragmentation	10,000		
	SYN_Flood	10,000		
	SynonymousIP_Flood	10,000		
	RSTFINFlood	10,000		
	TCP_Flood	10,000		
	ACK_Fragmentation	10,000		
	UDP_Fragmentation	10,000		
	SlowLoris	10,000		
	HTTP_Flood	10,000		
DoS	SYN_Flood	10,000	40,000	12.8%
	TCP_Flood	10,000		
	UDP_Flood	10,000		
	HTTP_Flood	10,000		
Mirai	udpplain	10,000	30,000	9.6%
	greip_flood	10,000		
	greeth_flood	10,000		
Benign	BenignTraffic	25,000	25,000	8%
Spoofing	DNS_Spoofing	10,000	20,000	6.4%
	MITM-ArpSpoofing	10,000		
Recon	PortScan	10,000	42,107	13.48%
	VulnerabilityScan	10,000		
	HostDiscovery	10,000		
	OSScan	10,000		
	PingSweep	2,107		
Web	SqlInjection	4,896	23,157	7.41%
	Backdoor_Malware	3,014		
	BrowserHijacking	5,453		
	XSS	3,584		
	CommandInjection	5,037		
	Uploading_Attack	1,173		
BruteForce	DictionaryBruteForce	12,191	12,191	3.9%
		Total	**312,455**	**100%**

Table 4: An example on how under sampling impacts the data imbalance. % stands for the portion of class rows in the dataset.

	rows B=15	% B=15	rows B=20	% B=20	rows B=75	% B=75
A	80	50%	80	76%	80	50%
B	15	15%	20	19%	75	47%
C	5	5%	5	5%	5	3%
Total	100	100%	105	100%	160	100%

To combat the data imbalance, we applied Random Under Sampling to the dataset to downsize the dominant classes. We sampled 10K random instances of each attack subcategory. For those subcategories that had less than 10K instances, we used all available instances. The BruteForce category had only one subcategory with almost 12K instances. We used all instances for this category. Benign did not have any subcategory, so initially, we sampled only 10K random instances of benign. However, our initial results showed low F1-score for benign category. Thus, we increased the number of benign samples to 25K. We tested higher numbers of benign instances as well, but 25K presented the best performance. Table 3 demonstrates the number of instances in each category in our dataset. This dataset is significantly more balanced than the original dataset, yet it still includes all attack categories and subcategories.

We created our models with higher numbers of benign instances as well. However, that declined the F1-socres for all classes. The reason is increasing the size of benign class while the sizes of minority classes remain the same reduces the portions of minority classes in the dataset.

Table 4 demonstrates an example of the way the class sizes in under sampling impact the data imbalance. Assume the dataset contains 3 classes A, B, and C with 80, 15, and 5 instances respectively. If we increase the number of class B instances to 75, the sizes of classes A and B will be almost the same, however the portion of class C in dataset declines from 5% to 3%, which means class C will be downsized by 40%. However, if we increase the class B's size to 20, it will be enlarged by 33% while the class C's portion remains almost the same. This makes a better balance between the impacts of classes. Class B in this example works the same way as the benign class in our dataset. Notice that in our dataset we cannot increase the sizes of minority classes because there is no more instance of those classes in the dataset. We also need to include a large number of instances from dominant classes (DDoS, DoS, and Mirai) because they have several subcategories, and we need enough instances of each subcategory to train the model.

Next, we split our dataset to 70% training and 30% test set. Then we created our models and tested them on this dataset. Our models demonstrated high F1-scores for dominant classes (DDoS, DoS, and Mirai) and relatively high F1-scores for other classes. We present our models and results in the next sections.

C. Class weights

Another technique to combat the data imbalance is class weighting. Most ML algorithms assume that the data is evenly distributed among classes. When the data distribution among classes is not balanced, the model tends to be more biased

towards predicting the minority classes because at the training time, the model does not have enough data to learn the patterns of these classes.

One way to reduce this negative impact is to assign different weights to the majority and minority classes during the training phase to balance their weights in the trained model. This technique aims to penalize the misclassifications of minority classes by assigning higher weights to minority classes and lower weights to majority classes to make a balance. However, assigning too high weights to minority classes may also reduce the model's performance. Thus, in an imbalanced dataset, we should find an appropriate arrangement of assigned weights that presents the best result. We used a grid search to find the weights assigned to the classes and achieved the best results when we assigned 0.5 weight to Mirai and 1 weight to other classes. Note that by default the weights of all classes are equal unless we manually assign weights to classes.

We also studied the impact of SMOTE oversampling to enlarge the minority classes but that did not change the results.

D. *Feature scaling*

Different features of the dataset have different scales and distributions. That reduces the performance of a model trained by this data. The main technique to address this issue is using feature scalers. Feature scaling is a technique that normalizes the range of each feature. We pre-processed our dataset with a variety of sklearn scalers. QuantileTransformer presented the best performance among different scalers. This scaler provides a non-linear transformation of data that shrinks the distances between the marginal outliers and inliers. It transforms the feature's distribution to a uniform distribution, which spreads out the most frequent values and reduces the impact of outliers.

E. *Feature selection*

Feature selection is a technique to identify a subset of features that together make a strong correlation with the label. It aims to minimize the noise on the output by eliminating the irrelevant features. We studied 3 feature selection techniques, containing (1) MDI [5], (2) Chi-square, and (3) ANOVA. We also studied the case that no feature selection technique is applied, meaning all features are used to train the model. [4] applied MDI feature selection in their ensemble learning model and selected the top 6 features selected by MDI, which are 'IAT', 'Magnitude', 'Total Size', 'Min', 'Flow Duration', and 'Total Sum'. We studied the same features. We applied Chi-sqaure to the dataset and selected the same number of features, containing 'Header_Length', 'Covariance' 'Rate', 'Srate', 'IAT', 'Total sum'. We also applied ANOVA to the dataset and selected the same number of features, containing 'Protocol Type', 'Variance, 'Min', 'Duration', 'ack_flag_number', and 'HTTPS'. We observe similarities between the top features selected by MDI and Chi-square, which are 'IAT', 'Total sum'. However, the features selected by ANOVA are completely different. Our studies showed that the performances of models with MDI and Chi-square are similar, and they are both better than ANOVA. We also trained our models with all features, and the results showed that such model presents the best performance, meaning a combination of all features has a strong correlation with the label. We will present the results and discussions in the next sections.

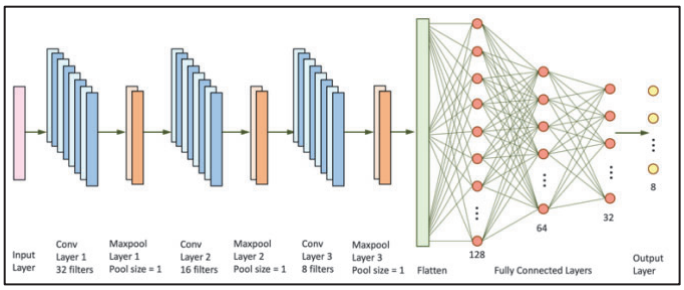

Fig 1. Model 1: 1D-CNN

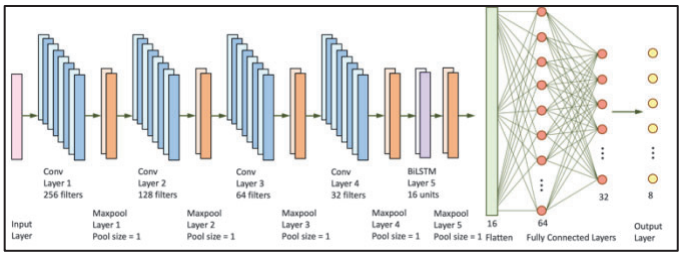

Fig 2. Model 2: BiLSTM-CNN

IV. The Proposed Deep Learning Models

We designed two DL models and trained them with our dataset for our network attack classification problem. We name our models model 1: 1D-CNN, and model 2: BiLSTM-CNN.

A. *Model 1: 1D-CNN*

Fig. 1. illustrates model 1. In this model, the input is in the form of time-series 1D sequence of data, and the output is a prediction of the class label. The class label is either benign or one of the 7 categories of attacks presented in Table 2. The model includes 3 convolution layers with 32, 16, 8 filters respectively, each followed by a maxpool of size 1. The convolution layers use kernel size 1 and ReLU activation function. The output of layer 3 is flattened to a 1D vector, which is fed into the first fully connected layer. There are 3 fully connected layers of 128, 64, and 32 neurons with ReLU activation functions. The output layer has 8 neurons, which is the number of classes, with softmax activation function, which produces the probability distribution over the class labels. The optimizer is adam and learning_rate is 0.001.

B. *Model 2: BiLSTM-CNN*

Fig. 2. illustrates model 2. The input and output layers are the same as model 1. This model includes 4 convolution layers with 256, 128, 64, 32 filters respectively, each followed by a maxpool of size 1. The convolution layers use kernel size 1 and ReLU activation function. These layers are followed by one BiLSTM layer with 16 units and Tanh activation function, followed by a maxpool of size 1. The output of layer 5 is flattened to a 1D vector, which is fed into the first fully connected layer. There are 2 fully connected layers of 64 and 32 neurons with ReLU activation functions. The output layer, optimizer, and learning rate are the same as model 1.

V. Results and Discussion

Table 5 illustrates the results of our models comparing to [2], which is the state-of-the-art on 8-class classifier DL model on our studied dataset.

Table 5: comparison of the proposed models with [2]

	Model	Feature selection	F1-score	Training time	Test time
1	CNN [2]	None	69.73	-	-
2	1D-CNN	ANOVA	65.36	133	7
3		MDI	89.69	144	4
4		Chi-square	90.57	130	4
5		**None**	**93.80**	**144**	**4**
6	BiLSTM-CNN	ANOVA	68.22	475	40
7		MDI	89.34	382	41
8		Chi-square	90.78	482	43
9		**None**	**93.82**	**970**	**40**

Table 6: Per-class results of 1D-CNN with all features

	Class	Precision	Recall	F1-score
1	DDoS	99.79	99.79	99.79
2	DoS	99.95	99.80	99.87
3	Mirai	100	99.62	99.81
4	Benign	84.30	90.48	87.28
5	Spoofing	89.36	79.77	84.29
6	Recon	87.40	85.45	86.41
7	Web	77.25	83.68	80.34
8	BruteForce	79.30	76.05	77.64

We present our results for 3 feature selection methods vs none where all features were used to train the models. The BiLSTM-CNN model with no feature selection outperforms the other models and the state-of-the-art [2] with 93.82% F1-score. The 1D-CNN model presents slightly less F1-score of 93.8% but significantly less training and test time. We select this model as our best model as it is computationally much more efficient.

We investigated the performance of our best model per each class. Table 6 illustrates per-class results. These results demonstrate that the model presents high F1-scores for DDoS, DoS and Mirai classes and comparable F1-scores in range 77-87% for other classes. For most classes, the precision and recall are the same or very close, which means the numbers of false positive and false negative predictions for the class are almost the same. High F1-scores in DDoS, DoS, and Mirai were expected as these are the largest classes of the dataset. Referring to the dataset features, we observe that most features are more TCP-based rather than UDP-based because the features are aggregated data extracted from a data stream between the source and the destination, for example the average or variance of packets lengths in the flow. In a TCP connection, usually there are several messages exchanged between the source and the destination in one data stream, thus, the aggregated data extracted from the data stream at the feature engineering phase is more informative comparing to a UDP or ICMP data stream, which usually contains only two messages, the request and the response. Thus, the aggregated data extracted from a UDP data stream provides limited information to identify the attack's pattern. For example, in Brute-force attack, the attacker aims to find out the user's password and sends several individual requests to the target machine where each attempts to log in with a different password. Each of these requests are typically sent through a separate data stream. In a spoofing attack, the attacker spoofs one message, and the target machine replies one response. That is also a short data stream. Most web-based attacks also do not involve many exchanged messages. For example, in SQL Injection the attacker injects one piece of SQL code into an input field and receives the response. For these types of attacks, non-aggregated features are more useful to learn the attack's pattern. For example, a combination of the protocol type, the length of packet, the duration of data stream, and some other related features creates patterns that our model learns to be able to classify these types of attacks. However, most of the dataset features are aggregated, so that the model is more successful in classifying TCP-based attacks, like most subcategories of DDoS, DoS, and Mirai. Our model improved the overall F1-score by 24% from 69.73% [2] to 93.8%. Also, our model provides almost balanced values for precision and recall for each class.

VI. Conclusion

In this research, we proposed two DL models to classify the network traffic to benign vs. 7 types of IoT attacks, containing DDoS, DoS, Recon, Web-Based, Brute Force, Spoofing, and Mirai. We used CICIoT2023 dataset, which has been created by conducting different subcategories of these 7 types of attacks on a large scale IoT network. This dataset is imbalanced. To combat the class imbalance, we applied random under sampling and class weighting techniques. To normalize the data distribution of each feature, we applied feature scaling. We studied the impact of feature selection techniques on the efficiencies our models. The results of our best performing model presented 24% improved F1-score comparing to the state-of-the-art. Finally, we analyzed the results and the way the nature of attack category impacts the model's performance in classifying the attack.

References

[1] CICIoT2023 dataset, https://www.unb.ca/cic/datasets/iotdataset-2023.html

[2] E.C. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, A.A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," Sensors, 2023, DOI: 10.3390/s23135941

[3] K.G. Narayan, S. Mookherji, V. Odelu, R. Prasath, A.C. Turlapaty, A.K. Das, "IIDS: Design of Intelligent Intrusion Detection System for Internet-of-Things Applications", arXiv preprint arXiv:2308.00943, 2023 Aug 2.

[4] R.W. Wardhani, D.S. Putranto, U. Jo, H. Kim, "Toward Enhanced Attack Detection and Explanation in Intrusion Detection System-Based IoT Environment Data," IEEE Access. 2023, DOI: 10.1109/ACCESS.2023.3336678

[5] H. Han, X. Guo, and H. Yu, ''Variable selection using mean decrease accuracy and mean decrease Gini based on random forest,'' IEEE ICSESS, 2016, DOI: 10.1109/ICSESS.2016.7883053

[6] S. Abbas, A. Al Hejaili, G.A. Sampedro, M. Abisado, A. Almadhor, T. Shahzad, K. Ouahada, "A Novel Federated Edge Learning Approach for Detecting Cyberattacks in IoT Infrastructures," IEEE Access, 2023, DOI: 10.1109/ACCESS.2023.3318866

[7] S. Abbas, I. Bouazzi, S. Ojo, A. Al Hejaili, G.A. Sampedro, A. Almadhor, M. Gregus, "Evaluating deep learning variants for cyber-attacks detection and multi-class classification in IoT networks," PeerJ Computer Science, 2024, DOI: 10.7717/peerj-cs.1793

[8] A.I. Jony, A.K. Arnob, "A long short-term memory based approach for detecting cyber attacks in IoT using CIC-IoT2023 dataset," Journal of Edge Computing, 2024, DOI: 10.55056/jec.648

[9] Z. Wang, H. Chen, S. Yang, X. Luo, D. Li, J. Wang, "A lightweight intrusion detection method for IoT based on deep learning and dynamic quantization," PeerJ Computer Science, 2023, DOI: 10.7717/peerj-cs.1569

[10] E.C. Nkoro, J.N. Njoku, C.I. Nwakanma, J.M. Lee, D.S. Kim, "Zero-Trust Marine Cyberdefense for IoT-Based Communications: An Explainable Approach," Electronics, 2024, DOI: 10.3390/electronics13020276